

Secure Your Code — Performance

Agent Implementation Specification

Autonomous execution mode

This document is written for unattended, end-to-end execution — implement it top to bottom without pausing for confirmation between steps or milestones.

Wherever this document says "verify," "check," "inspect," or "decide," resolve it yourself using available tools (read installed package docs/ `--help` output, run the compatibility probe in section 10.2, inspect the repository) and proceed with the most reasonable choice. Record every such decision in `docs/architecture.md` as you go, so choices are auditable after the fact — but do not stop and ask before making them. Concretely, unless told otherwise:

- **Graphify package/fork:** pin `Graphify-Labs/graphify` (or its published Python package) unless the environment already has a different Graphify installation, in which case use that one and document it. Whichever package gets pinned, its exact MCP tool surface must still be enumerated via the compatibility probe (section 10.2) — different actively-maintained Graphify variants expose meaningfully different tool sets, not just renamed equivalents, so do not assume the illustrative tool names used elsewhere in this document (`query_graph` , `get_node` , etc.) exist on the pinned package.

- **Copilot model string:** use "auto" unless a specific model is already configured elsewhere in the environment.
- **Compatibility probe (section 10.2) outcome:** if it passes, proceed with the full custom-agent design as written. If it fails, apply the stated fallback (heavier pre-injection of data into specialists, per item 1 in section 10) and continue – do not halt the build waiting for a decision.
- **Ambiguous implementation details not otherwise specified:** choose the simplest option consistent with the constraints in section 21 and the non-goals in section 20, implement it, and note the choice.

Only stop and surface a question to a human if execution hits a genuinely blocking condition with no defined resolution in this document – for example, a missing required credential/secret, an action that would affect something outside the configured repository (`RepoPath`), or a compatibility probe failure with no viable fallback path. Everything else: decide, document, and keep going.

Objective

Implement a production-oriented **.NET performance static-analysis agent** using the GitHub Copilot SDK for .NET, Graphify MCP, and a custom Roslyn analyzer.

The agent must investigate a repository for:

- potential memory retention / leaks
- allocation pressure and GC pressure
- CPU bursts / CPU amplification
- excessive concurrency and ThreadPool pressure

- database and HTTP amplification
- serialization-related overhead
- resource lifetime problems
- other high-impact production performance risks

The implementation must distinguish **static-analysis hypotheses** from **verified runtime problems**. Do not claim that a memory leak, CPU burst, or production incident is confirmed unless supporting runtime/benchmark evidence exists.

Important design decision

Use the tools for different jobs:

Component	Responsibility
GitHub Copilot SDK	Agent orchestration, reasoning, tool calls, report generation
Graphify MCP	Repository topology, symbol relationships, call paths, architectural context
Built-in .NET/Roslyn analyzers	Existing deterministic diagnostics
SecureYourCode custom Roslyn analyzer	Project-specific performance anti-pattern detection
Verification tools	Tests, benchmarks, runtime diagnostics

Component	Responsibility
Copilot verifier/critic	Challenge findings and search for counter-evidence

Do not use CodeQL in this project. The project should remain based on components suitable for the intended commercial/offline environment.

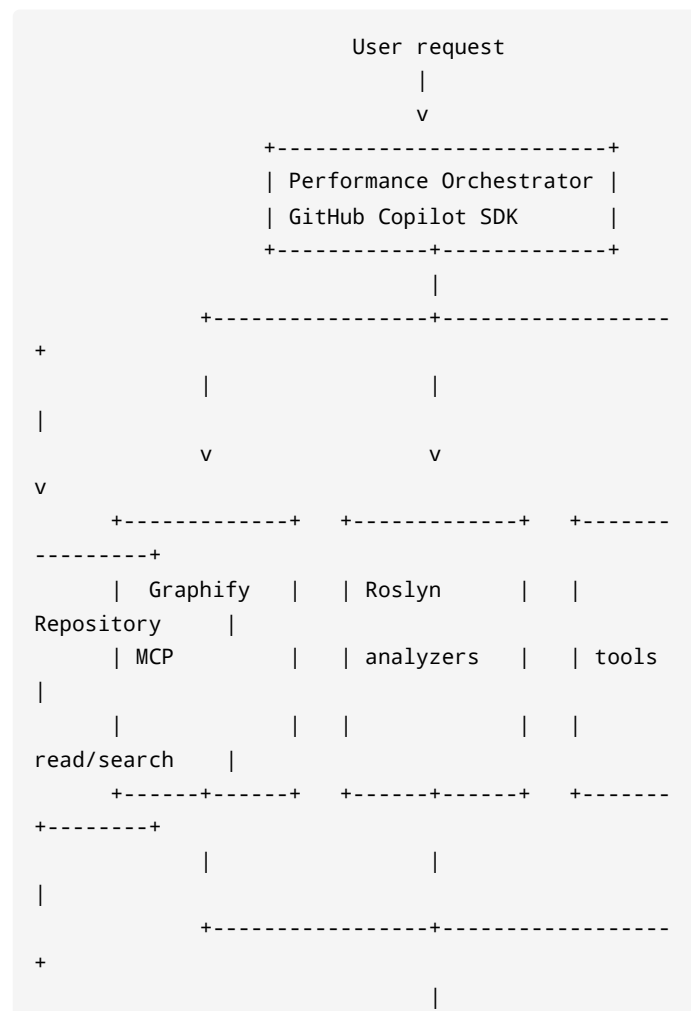
Scope decision – v1 targets exactly one repository. The agent is a local companion service, co-located on the same machine as the git working copy it analyzes – the same repository the git post-commit hook fires from (section 4.4). It does not analyze arbitrary external repositories in a single running instance. The repository root is read once from configuration (`RepoPath` , section 4.4) and used consistently by the Graphify graph, the git hook, the background refresh worker, and the default target of the analysis endpoint (section 10.1). `run_static_analysis` 's `path` / `profile` inputs (section 9) select a project/solution *within* that configured root, not a different repository. Multi-repository support is a deliberate non-goal for v1 (section 20).

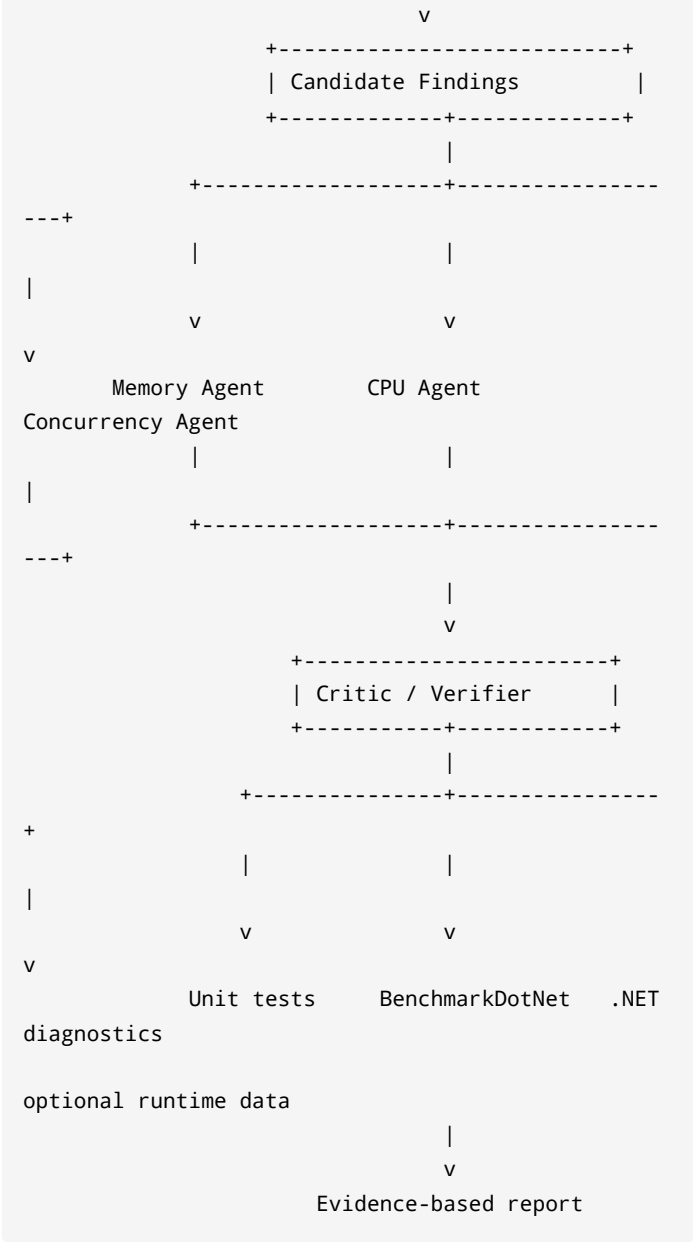
Graphify currently exposes graph traversal through MCP, including tools such as `query_graph` , `get_node` , `get_neighbors` , and `shortest_path` . Graphify's current documentation also states that code extraction is local and AST-based. [Graphify](#).

The GitHub Copilot SDK supports custom agents, MCP servers, tools, skills, and parallel sub-agent orchestration. [Copilot SDK](#) [Copilot MCP](#)

Roslyn custom analyzers are implemented through `DiagnosticAnalyzer` and related compiler-analysis APIs. [Microsoft Roslyn SDK tutorial](#)

1. Target architecture





2. Repository structure

Create or adapt the repository to the following structure:

```
SecureYourCode/  
|  
+-- .husky/  
|   +-- task-runner.json  
|   +-- post-commit  
|  
+-- src/  
|   +-- SecureYourCode.Agent/  
|   (ASP.NET Core (.NET 10) web application, minimal  
|   API)  
|   |   +-- Program.cs  
|   |   +-- Agents/  
|   |   |   +-- PerformanceOrchestrator.cs  
|   |   |   +-- MemoryReviewer.cs  
|   |   |   +-- CpuReviewer.cs  
|   |   |   +-- ConcurrencyReviewer.cs  
|   |   |   +-- VerificationReviewer.cs  
|   |   +-- Tools/  
|   |   |   +-- StaticAnalysisTool.cs  
|   |   |   +-- VerificationTool.cs  
|   |   |   +-- RepositoryTool.cs  
|   |   +-- Graph/  
|   |       +-- GraphRefreshEndpoint.cs    (POST  
|   |       /git-post-commit)  
|   |       +-- GraphRefreshWorker.cs  
|   |       (background channel consumer)  
|   |       +-- GraphifyUpdater.cs        (first-  
|   |       extract vs update, atomic write, metadata)  
|   |  
|   +-- SecureYourCode.PerformanceAnalyzer/  
|   |   +-- SecureYourCodePerformanceAnalyzer.cs  
|   |   +-- Rules/  
|   |       +-- DbCallInsideLoopAnalyzer.cs
```

```

| |      +-- HttpCallInsideLoopAnalyzer.cs
| |      +-- UnboundedTaskWhenAllAnalyzer.cs
| |      +-- UnboundedCollectionAnalyzer.cs
| |      +-- BlockingAsyncAnalyzer.cs
| |      +-- AllocationInsideLoopAnalyzer.cs
| |
| +-- SecureYourCode.PerformanceAnalyzer.Tests/
|     +-- DbCallInsideLoopAnalyzerTests.cs
|     +-- HttpCallInsideLoopAnalyzerTests.cs
|     +-- ...
|
+-- prompts/
|     +-- performance-orchestrator.md
|     +-- memory-reviewer.md
|     +-- cpu-reviewer.md
|     +-- concurrency-reviewer.md
|     +-- verifier.md
|
+-- test-assets/
|     +-- seeded-performance-bugs/
|     +-- false-positives/
|
+-- docs/
|     +-- architecture.md
|     +-- rules.md
|     +-- verification.md
|
+-- .editorconfig
+-- Directory.Build.props
+-- Directory.Packages.props    (only if repository
                                already uses central package management)
+-- README.md

```

Adapt names if the existing project already has an established structure. Do not restructure unrelated application code just to match this document.

3. Phase 1 — repository reconnaissance

The agent must first establish:

- target framework(s)
- solution/project structure
- ASP.NET Core endpoints/controllers/minimal APIs
- gRPC entry points
- background services / `BackgroundService`
- queue/message consumers
- scheduled jobs/timers
- database access
- HTTP clients
- serialization libraries
- caches
- concurrency primitives
- singleton/scoped/transient registrations
- relevant configuration files

The orchestrator should not immediately ask an LLM to read every source file. This is enforced at the framework level via `DefaultAgent.ExcludedTools` (section 10), which hides deep analyzer/graph tools from the default agent and forces delegation to specialist sub-agents.

First use Graphify to understand the repository graph.

4. Graphify integration

4.1 Build/update graph

The agent should ensure a current Graphify graph exists before analysis.

Pin one specific Graphify package/fork explicitly (record the exact PyPI/crate name and version in `docs/architecture.md`) before writing any code against it. Several actively-maintained Graphify implementations exist with meaningfully different CLI conventions — some use `update` as a subcommand, others as a flag on the main invocation (`graphify . --update`) — so treat every command below as illustrative, not authoritative:

```
graphify extract .  
graphify update .
```

and MCP serving from the generated graph, for example:

```
python -m graphify.serve graphify-out/graph.json
```

Do not implement against the snippets above without first running the installed package's own `--help` and reading its `USAGE.md` (or equivalent). This is not optional polish — it's a required step, because the exact subcommand/flag shape depends entirely on which package/fork got pinned.

The Graphify repository documents a local AST-based extraction model and MCP tools including `query_graph`, `get_node`, `get_neighbors`, and `shortest_path`. [Graphify](#)

4.2 MCP configuration

Configure Graphify as a **read-only MCP server** in the Copilot session.

Do not expose unnecessary Graphify tools. Prefer an allow-list such as:

```
query_graph
get_node
get_neighbors
shortest_path
```

The Copilot SDK supports local/stdio MCP servers and explicit MCP tool allow-lists. [GitHub Copilot SDK MCP documentation](#)

For .NET the current SDK configuration uses `McpServers` and `McpStdioServerConfig`, for example:

```
using GitHub.Copilot;

await using var client = new CopilotClient();
await using var session = await
client.CreateSessionAsync(new SessionConfig
{
    Model = "gpt-5",
    OnPermissionRequest =
PermissionHandler.ApproveAll,
    McpServers = new Dictionary<string,
McpServerConfig>
    {
        ["graphify"] = new McpStdioServerConfig
        {
            Command = "python",
            Args = new List<string>
            {
                "-m",
                "graphify.serve",
                "<absolute-path>/graphify-
```

```

out/graph.json"
    },
    Tools = new List<string>
    {
        "query_graph",
        "get_node",
        "get_neighbors",
        "shortest_path"
    }
}
}
});

```

Note the namespace is `GitHub.Copilot`, not `GitHub.Copilot.SDK` — the NuGet package name and the C# namespace differ.

`OnPermissionRequest` is required for an unattended agent. Tool and MCP calls go through a permission gate by default; without a handler, the session hangs waiting for approval on the first tool call. Use `PermissionHandler.ApproveAll` for this unattended analysis agent, or a narrower custom handler if finer-grained control is later needed.

Verify the exact class/property names against the installed `GitHub.Copilot.SDK` package before implementation because the SDK is evolving.

The `<absolute-path>/graphify-out/graph.json` argument above should be resolved from the `GraphOutputPath` configuration value (section 4.4), not hard-coded — this keeps the MCP server, the git-hook refresh worker, and the analysis endpoint all pointing at the same file.

4.3 Required Graphify queries

Create reusable query templates for:

Production entry points

Identify application execution entry points including ASP.NET endpoints, controllers, minimal APIs, gRPC handlers, message consumers, BackgroundService implementations, scheduled jobs and timers.

Hot execution paths

Identify execution paths from production entry points involving:

- database access
- HTTP calls
- serialization/deserialization
- loops
- allocations
- caching
- channels and queues
- Task/async operations
- locks/semaphores
- parallel execution

Highlight long call chains, high fan-out methods, highly connected nodes, repeated operations, and paths with several amplification mechanisms.

Memory retention paths

Find long-lived components such as static objects, singleton services,

BackgroundService instances and caches that retain collections, tasks, subscriptions, buffers or request-related objects. Trace the ownership/retention path where possible.

CPU amplification paths

Find production-reachable paths containing nested loops, repeated enumeration, database/HTTP calls in loops, serialization in loops, large task fan-out, Parallel operations, retry loops, polling or timers.

Concurrency paths

Find Task.WhenAll, Parallel.*, Channel, SemaphoreSlim, locks, producer/consumer paths, background workers and potentially unbounded concurrency. Identify whether the input or concurrency limit is bounded.

4.4 Automatic graph refresh via git hooks

Keep the Graphify graph current automatically on every commit, without slowing down `git commit` or duplicating decision logic in a shell script. The hook only fires a signal; the ASP.NET application owns all the logic.

Read the repository root, the Graphify graph path, and the listen port from configuration rather than hard-coding them — the same `RepoPath` value is used by the Graphify graph, the git

hook target, and the default target of `/analyze` (section 10.1), per the single-repository scope decision above:

```
// appsettings.json
{
  "SecureYourCode": {
    "RepoPath": "/absolute/path/to/repo",
    "GraphOutputPath":
      "/absolute/path/to/repo/graphify-out/graph.json"
  }
}
```

```
// launchSettings.json / hosting config – keep
this port in sync with the curl
// target in .husky/task-runner.json below; do not
let the two drift independently
{
  "applicationUrl": "http://127.0.0.1:9876"
}
```

Husky.Net setup (once per clone, automatic via MSBuild):

```
dotnet new tool-manifest # if not already
present
dotnet tool install Husky
dotnet husky install
```

Add to `SecureYourCode.Agent.csproj` so hooks (re)install automatically on build/restore for every clone, without a manual setup step:

```
<Target Name="husky"
  BeforeTargets="Restore;CollectPackageReferences"
  Condition="'$(HUSKY)' != 0">
  <Exec Command="dotnet tool restore"
```

```

StandardOutputImportance="Low"
StandardErrorImportance="Low" />
  <Exec Command="dotnet husky install"
StandardOutputImportance="Low"
StandardErrorImportance="Low"
WorkingDirectory="$(MSBuildProjectDirectory)" />
</Target>

```

`.husky/post-commit:`

```

#!/bin/sh
. "$(dirname "$0")/_/husky.sh"
dotnet husky run --name graph-refresh

```

`.husky/task-runner.json:`

```

{
  "$schema":
    "https://alirezanet.github.io/Husky.Net/schema.json",
  "tasks": [
    {
      "name": "graph-refresh",
      "group": "post-commit",
      "command": "curl",
      "args": ["-s", "-X", "POST",
        "http://127.0.0.1:9876/git-post-commit"]
    }
  ]
}

```

The task does nothing but fire the request. No branching or extraction logic lives in the hook script itself.

Register the refresh channel as a singleton in `Program.cs`
`(builder.Services.AddSingleton(Channel.CreateBounded<bool>`

```
(new BoundedChannelOptions(1) { FullMode =  
BoundedChannelFullMode.DropWrite }))) so it can be  
injected into both the endpoint and the background worker  
below.
```

Non-blocking endpoint (`Graph/GraphRefreshEndpoint.cs`): enqueues a refresh request and returns immediately, so `curl` (and therefore the commit) is never held up waiting for `graphify` to run:

```
app.MapPost("/git-post-commit", (Channel<bool>  
refreshQueue) =>  
{  
    // capacity 1 + DropWrite: a burst of commits  
    collapses into at most one pending refresh  
    refreshQueue.Writer.TryWrite(true);  
    return Results.Accepted();  
});
```

Background worker (`Graph/GraphRefreshWorker.cs`), the single consumer of the queue:

```
public class GraphRefreshWorker :  
    BackgroundService  
{  
    protected override async Task  
    ExecuteAsync(CancellationToken stoppingToken)  
    {  
        await foreach (var _ in  
            _refreshQueue.Reader.ReadAllAsync(stoppingToken))  
        {  
            try  
            {  
                // reads HEAD at execution time,  
                not at trigger time, so a burst of  
                // commits correctly resolves to
```

```

the latest commit, not a stale one
        await
_graphifyUpdater.RefreshAsync(_repoPath,
stoppingToken);
    }
    catch (Exception ex)
    {
        _logger.LogWarning(ex, "graphify
refresh failed"); // logged separately from
analysis findings – section 21
    }
}
}
}

```

GraphifyUpdater.RefreshAsync

(`Graph/GraphifyUpdater.cs`) is the single source of truth for the refresh decision – called by the worker above, by the startup check, and by the analysis-start check below. It:

1. Checks whether `graphify-out/graph.json` exists → runs the pinned package's full-extract command (first time) or its incremental-update command (subsequent), per section 4.1.
2. **Handles a shrink-guard refusal without blindly forcing through it.** Some Graphify variants refuse to overwrite an existing graph when a rebuild would produce fewer nodes than it started with, and offer a force-override flag/parameter – a legitimate refactor or file deletion can trigger this refusal. Documented incidents show blindly forcing past this guard is not safe in general: it has caused a fuzzy-deduplication pass to silently collapse legitimate, still-existing nodes as if they were duplicates, destroying real graph data rather than just removing genuinely deleted symbols. On a shrink-guard refusal:

- if the pinned package exposes a dedicated deletion/rename-handling operation (some do — a "prune" style operation that removes nodes only for files actually gone from the working tree, verified via the compatibility probe in section 10.2), use that instead of forcing the update through;
 - otherwise, do not auto-force. Log the refusal, keep the last-known-good graph in place, and surface the staleness in the report (section 16) rather than risking a silent, destructive overwrite.
3. Writes graph output to a temp file and atomically renames it into place, so a session that starts mid-refresh never reads a partial file.
 4. **Validates the output before treating the refresh as successful** — the temp file must parse as valid JSON and be non-empty — before it is renamed into place and before `graph.meta.json` is touched. A failed or garbage `graphify` run must never cause the metadata to claim the graph is current.
 5. Writes `graphify-out/graph.meta.json` with the commit SHA the graph was built from, whether the working tree was dirty at build time (`git status --porcelain non-empty`), and a build timestamp — so freshness can be checked cheaply and so the report (section 16) can state which commit the analysis is based on and whether it reflects uncommitted local changes.

Self-healing freshness checks: call

`GraphifyUpdater.RefreshAsync` (not just the queue) once at application startup and again at the start of the analysis flow, comparing the stored SHA in `graph.meta.json` to current `HEAD`. This covers the case where the hook never fired — the

app wasn't running, `curl` hit connection refused, or the hook wasn't installed yet — so the graph self-heals instead of silently staying stale.

Graphify MCP process lifetime: because the graph can be refreshed at any time, do not run `graphify serve` as one long-lived shared process that loads the graph once into memory. Let the Copilot SDK spawn it fresh, per the local/stdio MCP config in section 4.2, for each analysis session, so it reads the current `graph.json` off disk at session start rather than needing an explicit restart.

5. Phase 2 — built-in .NET/Roslyn analysis

Use the analyzers shipped with the installed .NET SDK where available.

Do not add `Microsoft.CodeAnalysis.NetAnalyzers` automatically if the project already gets the standard analyzers from the .NET SDK. Respect the repository's existing analyzer/package setup.

Run analysis through the project build or a dedicated analyzer command.

Example baseline:

```
dotnet build <solution-or-project> --no-restore -v:minimal
```

Do not treat every compiler warning as a performance finding. Filter and classify diagnostics relevant to memory, allocation, performance, concurrency, async, disposal, or database behavior.

Create a structured result model for diagnostics rather than passing large raw build logs into the LLM.

6. SecureYourCode custom Roslyn analyzer

6.1 Purpose

Create a standalone analyzer project:

```
SecureYourCode.PerformanceAnalyzer
```

It must implement Roslyn `DiagnosticAnalyzer` rules.

A Roslyn analyzer is deterministic compiler-aware static analysis. It should identify a source-code pattern and report a diagnostic. It should **not** claim that the pattern proves a production incident.

Microsoft's Roslyn SDK documents the `DiagnosticAnalyzer` model, analysis callbacks, syntax/semantic analysis, and diagnostic reporting. [Microsoft Roslyn SDK](#)

6.2 Initial rules

Implement the following rules first.

ID	Rule	Initial severity	Goal
PERF001	DB call inside loop	Warning	Detect likely N+1/amplification
PERF002	HTTP call inside loop	Warning	Detect network amplification
PERF003	Unbounded <code>Task.WhenAll</code>	Warning	Detect excessive task fan-out
PERF004	Potential unbounded collection/cache	Warning	Detect possible memory growth
PERF005	Blocking async path	Warning	Detect <code>.Result</code> , <code>.Wait()</code> , blocking APIs in async execution
PERF006	Expensive allocation inside loop	Info/Warning	Detect repeated allocations on loops

Do not implement broad fuzzy rules. Every diagnostic must have a clear syntax/semantic basis.

7. Rule details

PERF001 — DB call inside loop

Detect database/repository operations called inside:

- `for`
- `foreach`
- `while`
- `do`
- LINQ transformations where the database call occurs per item

Initial target patterns may include methods such as:

```
IQueryable / Entity Framework query execution  
SaveChanges / SaveChangesAsync  
repository methods  
Dapper query methods  
known application DB abstractions
```

Do not hard-code one application-specific repository interface.
Prefer configurable matching using method/type naming plus optional configuration.

Diagnostic should contain:

```
PERF001  
Database operation appears inside an iteration  
construct; potential query amplification.
```

PERF002 – HTTP call inside loop

Detect calls to known HTTP APIs inside iteration constructs, initially including:

```
HttpClient.Send  
HttpClient.SendAsync  
HttpClient.GetAsync  
HttpClient.PostAsync
```

```
HttpClient.PutAsync  
HttpClient.DeleteAsync
```

Consider common wrapper abstractions later.

PERF003 — Unbounded Task.WhenAll

Detect patterns where an enumerable of unknown/unbounded size is converted into a large task collection and passed to

```
Task.WhenAll .
```

Examples of interest:

```
await Task.WhenAll(items.Select(ProcessAsync));
```

where `items` can represent external/request-sized or otherwise unbounded input.

Do not warn on clearly bounded fixed-size arrays unless there is another reason to suspect amplification.

PERF004 — Potential unbounded collection/cache

Look for fields in long-lived types that store data into collections without an obvious bound.

High-interest patterns:

```
static collection  
singleton-owned collection  
ConcurrentDictionary  
Dictionary  
List  
HashSet
```

```
Channel.CreateUnbounded  
IMemoryCache without obvious expiration/size  
policy
```

This rule must use conservative wording:

```
Potential unbounded state/collection. Review  
ownership, eviction,  
expiration and capacity limits before classifying  
as memory retention.
```

Do not label this rule "memory leak".

PERF005 — Blocking async path

Detect, where semantically identifiable:

```
Task.Result  
Task.Wait()  
WaitHandle.WaitOne  
Thread.Sleep in async server execution  
GetAwaiter().GetResult()
```

Report potential ThreadPool starvation / throughput degradation. Do not claim starvation is proven.

PERF006 — Expensive allocation inside loop

Start conservatively with clear patterns such as:

```
new large arrays/buffers  
new large strings through repeated concatenation
```

```
JSON serialization inside loops
known expensive object creation inside loops
```

Use syntax/semantic analysis and avoid speculative allocation warnings.

8. Analyzer diagnostic metadata

Each diagnostic should use a stable descriptor with:

- ID
- title
- message format
- category = Performance
- default severity
- enabled by default = true
- help link to local/project documentation

Add unit tests using the standard Roslyn analyzer testing infrastructure.

Each rule must have:

1. positive test cases
2. negative/valid cases
3. boundary cases
4. multiple syntactic forms where reasonable
5. regression tests for every bug discovered during development

No code fix is required for the first version. The agent should provide remediation advice rather than automatically editing

source code.

9. Static analysis tool exposed to Copilot

Implement a custom Copilot tool named:

```
run_static_analysis
```

Inputs:

```
path          – a project/solution within the  
configured RepoPath (section 4.4); not an  
arbitrary external repository, per the single-  
repository scope decision  
configuration/profile (optional)
```

The tool should:

1. locate solution/project
2. run build/analyzers
3. collect standard analyzer diagnostics
4. collect SecureYourCode PERFxxx diagnostics
5. normalize results
6. return structured JSON

Do not return enormous raw compiler logs unless explicitly requested.

Recommended output model:

```
{
  "repository": "...",
  "solution": "...",
  "succeeded": true,
  "diagnostics": [
    {
      "id": "PERF001",
      "category": "Database",
      "severity": "warning",
      "file": "OrderService.cs",
      "line": 127,
      "column": 17,
      "message": "Database operation appears
inside an iteration construct; potential query
amplification."
    }
  ]
}
```

The tool should include analyzer execution duration and exit code for debugging, but avoid exposing irrelevant build noise to the model.

9.1 Tool registration

Register the tool with the SDK's function-factory pattern (`Microsoft.Extensions.AI`), not a bespoke tool class:

```
using Microsoft.Extensions.AI;
using System.ComponentModel;

var runStaticAnalysis = CopilotTool.DefineTool(
    (
        [Description("Path to the solution or
project to analyze")] string path,
        [Description("Optional analyzer
```

```
configuration/profile name"")] string? profile
) =>
{
    // locate solution/project, run dotnet
    build + PERFxxx analyzers,
    // normalize results into the diagnostics
    model above
    return new { repository = path, solution =
    "...", succeeded = true, diagnostics = new
    object[] { } };
},
factoryOptions: new AIFunctionFactoryOptions
{
    Name = "run_static_analysis",
    Description = "Runs built-in .NET/Roslyn
    analyzers and SecureYourCode PERFxxx analyzers and
    returns structured diagnostics.",
});
```

Add `runStaticAnalysis` to the orchestrator session's `Tools` list. Verify the exact `CopilotTool / AIFunctionFactoryOptions` surface against the installed SDK version before implementation.

10. Copilot orchestration

Use the GitHub Copilot SDK .NET package:

```
GitHub.Copilot.SDK
```

The current SDK repository documents .NET support and the `GitHub.Copilot.SDK` package. [SDK repository](#).

The main orchestrator's *session* has repository read/search tools, `run_static_analysis`, the Graphify MCP tools, and verification tools all configured and available somewhere in the session – but the main/default agent does not call the heavy ones directly. Per the `DefaultAgent.ExcludedTools` configuration below, `run_static_analysis` and the Graphify MCP tools are deliberately excluded from the *default* agent and delegated to the specialist sub-agents, which is where the actual analysis happens.

Graphify should be read-only.

Before delegating to specialists, have the `IOrchestrator C# code` – not the default/root LLM agent – call the underlying `run_static_analysis` function directly (a plain method call, outside the SDK's tool-calling path entirely) and inject the resulting structured diagnostics directly into each specialist's initial prompt/context, in addition to leaving the tool available to them via `specialistTools` below. This is deliberate redundancy, not wasted work: current Copilot SDK issue history documents cases where a custom sub-agent's declared tools are echoed back as available in session events but the model never actually calls them, with no error surfaced – work silently doesn't happen. Pre-injecting the diagnostics means a specialist that fails to invoke the tool live still has the data in front of it. This pre-computation step is unrelated to, and does not conflict with, `DefaultAgent.ExcludedTools` below – that setting governs what the *default LLM agent* can call as a tool during the conversation; it says nothing about the orchestration code that runs before the conversation starts.

The orchestrator should delegate independent analysis to specialist agents using the SDK's native `CustomAgentConfig` mechanism – a single session with agent definitions, rather

than manually creating and coordinating separate sessions per specialist. The runtime auto-delegates to a sub-agent by matching the user's request against each agent's `Description`, so descriptions must be specific.

Recommended specialists, defined as `CustomAgentConfig` entries on the orchestrator's `SessionConfig`:

```
// Shared by all analysis specialists: they must
// be able to query the graph
// and pull structured diagnostics, or they cannot
// do the job they're delegated.
var specialistTools = new List<string>
{
    "grep", "glob", "view",
    "query_graph", "get_node", "get_neighbors",
    "shortest_path",
    "run_static_analysis",
};

CustomAgents = new List<CustomAgentConfig>
{
    new()
    {
        Name = "MemoryReviewer",
        Description = "Investigates static/global
state, singleton lifetime, unbounded collections,
cache eviction, event/subscription retention, and
other memory-retention paths. Read-only.",
        Tools = specialistTools,
        Prompt = "You are a memory-retention
specialist. Analyze code and Graphify paths for
the categories in section 11. Never modify files.
Distinguish retention from normal long-lived
memory.",
    },
    new()
```

```

{
    Name = "CpuReviewer",
    Description = "Investigates CPU
amplification: nested loops, repeated enumeration,
DB/HTTP calls in loops, serialization in loops,
task fan-out, and retry/polling patterns. Read-
only.",
    Tools = specialistTools,
    Prompt = "You are a CPU-amplification
specialist. Focus on production-reachable
amplification, not isolated micro-optimizations.
Never modify files.",
},
new()
{
    Name = "ConcurrencyReviewer",
    Description = "Investigates unbounded
concurrency, Task.WhenAll/Parallel APIs, channels,
queues, locks, ThreadPool usage, and retry storms.
Read-only.",
    Tools = specialistTools,
    Prompt = "You are a concurrency
specialist. Identify whether concurrency/input is
bounded. Never modify files.",
},
new()
{
    Name = "VerificationReviewer",
    Description = "Challenges strong candidate
findings, searches for counter-evidence, and
proposes or generates reproducible
tests/benchmarks.",
    Tools = specialistTools.Concat(new[] {
"bash" }).ToList(), // bash needed to run
generated tests/benchmarks
    Prompt = "You are the critic/verifier. For
each strong candidate, attempt to disprove it,
search for counter-evidence (see section 12), and

```

```

propose or generate a reproducible test or
BenchmarkDotNet benchmark.",
    Infer = false, // invoked explicitly by
the orchestrator after candidates are
consolidated, never auto-selected
    },
},

```

Notes:

- `specialistTools` gives every analysis specialist the Graphify query tools and `run_static_analysis` — this is what makes delegation actually work: excluding these from the default agent (below) only makes sense because the specialists have them instead. An agent's `Tools` list is exhaustive, not additive to whatever the default agent has — omitting a tool here means that agent cannot call it, full stop.
- `Tools` deliberately excludes `edit / bash` for the three analysis reviewers — this enforces "never modify application source during analysis" (section 21) at the framework level, not only in the prompt.
`VerificationReviewer` is the one exception, since it needs `bash` to run generated tests/benchmarks. Because a prompt instruction alone is weaker than the framework-level enforcement used everywhere else, wrap `VerificationReviewer`'s invocation in a tracked-file integrity check: `snapshot git status --porcelain` before it runs and compare after; if any tracked file changed, flag the run and discard its findings rather than silently trusting that "never modify files" was followed.
- `Infer = false` on `VerificationReviewer` means the runtime never auto-selects it from user intent; the orchestrator invokes it explicitly once candidates are

consolidated, matching the "every finding must pass a challenge step" rule in section 12.

- Use `DefaultAgent.ExcludedTools` on the orchestrator session to hide `run_static_analysis` and the Graphify MCP tools from the main/default agent, forcing it to delegate to the specialist sub-agents for anything that reads deep analyzer/graph output. This is the concrete mechanism for the Phase 1 rule "the orchestrator should not immediately ask an LLM to read every source file" (section 3):

```
DefaultAgent = new DefaultAgentConfig
{
    ExcludedTools = new List<string> {
        "run_static_analysis", "query_graph", "get_node",
        "get_neighbors", "shortest_path" },
    },
```

- Subscribe to `subagent.started` / `subagent.completed` / `subagent.failed` events on the orchestrator session and record `agentDisplayName`, `durationMs`, and `totalToolCalls` for each specialist run. Fold this into the final report (section 16) as an execution/traceability trail — "who found what, and how long it took" — rather than discarding it.

Custom agents have their own prompts/tool scopes and can operate as sub-agents. [Copilot custom agents](#)

10.1 Analysis entry point

Expose one HTTP entry point on the ASP.NET host that runs the pipeline described above and returns the final report:

```

app.MapPost("/analyze", async (AnalyzeRequest?
request, IOrchestrator orchestrator,
Cancellation token ct) =>
{
    if (!orchestrator.TryEnterAnalysisGate())
    {
        return Results.Conflict("An analysis is
already running. Wait for it to complete before
starting another.");
    }
    try
    {
        // path defaults to the configured
RepoPath (section 4.4) per the single-repository
        // scope decision above; an explicit path
selects a project/solution within that root
        var repoRoot = request?.Path ??
configuredRepoPath;
        var report = await
orchestrator.RunAnalysisAsync(repoRoot,
request?.Profile, ct);
        return Results.Ok(report);
    }
    finally
    {
        orchestrator.ExitAnalysisGate();
    }
});

public record AnalyzeRequest(string? Path, string?
Profile);

```

TryEnterAnalysisGate / ExitAnalysisGate wrap a simple single-slot gate (e.g. SemaphoreSlim(1, 1) with a non-blocking Wait(0)) so a second concurrent request is rejected

outright rather than allowed to run a competing pipeline, MCP subprocess, and graph refresh alongside the first.

For v1, keep this synchronous: one request, one full pipeline run (Graphify freshness check, specialist delegation, critic, verification), one report in the response, with a generous server-side timeout to cover LLM latency. A job-submission/polling or streaming (SSE) version that surfaces `subagent.*` lifecycle events as they happen is a natural upgrade once the synchronous path works, but is not required for v1.

Before running the pipeline, call

`GraphifyUpdater.RefreshAsync` (section 4.4) as the analysis-start freshness check, so a request is never served against a graph the hook failed to update.

`IOrchestrator.RunAnalysisAsync` is responsible for composing one `SessionConfig` from every piece defined above — `Model`, `OnPermissionRequest`, `McpServers` (section 4.2), the `run_static_analysis` tool (section 9.1), `CustomAgents`, and `DefaultAgent.ExcludedTools` (section 10) — and calling `CreateSessionAsync` with it. **Create a new session per `/analyze` call rather than holding one shared session for the lifetime of the app.** This is not just a lifecycle preference: it's what makes the "spawn the Graphify MCP server fresh per session" guarantee in section 4.4 actually true. A singleton session created once at startup would keep its first-loaded `graph.json` for the app's entire lifetime, silently defeating the whole git-hook refresh mechanism.

10.2 Compatibility probe (run before building the rest of this section)

The custom-agent/tool-delegation design above depends on specific, currently-evolving SDK behavior. Before implementing all four specialists, `DefaultAgent.ExcludedTools`, and per-agent MCP wiring, run a minimal smoke test against the actually-installed SDK version and record the result in `docs/architecture.md`:

1. Create one session with one registered custom tool (a trivial `CopilotTool.DefineTool` stub) and one `McpStdioServerConfig` (the pinned Graphify package/fork from section 4.1).
2. Define one `CustomAgentConfig` whose `Tools` list references that custom tool and an MCP tool by name.
3. Exclude the custom tool from the default agent via `DefaultAgent.ExcludedTools`.
4. Prompt the session so the runtime delegates to the custom sub-agent, and confirm — via `subagent.started` / `tool.execution_start` events or an observable side effect, not just the model's claimed response text — that the sub-agent actually invoked both the custom tool and the MCP tool, not merely that they were listed as available.
5. **Enumerate the pinned Graphify package's actual exposed MCP tools before wiring any `Tools` /allow-list in this document.** Different actively-maintained Graphify variants expose meaningfully different tool surfaces — not just renamed equivalents of `query_graph` / `get_node` / `get_neighbors` / `shortest_path`, but entirely different capability sets (e.g. some expose freshness-checking, diffing, and prune-style deletion-handling tools instead of, or in addition to, raw graph traversal). Do not assume the four tool names used

elsewhere in this document (section 4.2, section 10) exist on the pinned package — confirm what it actually returns, and update every reference in this document to match reality rather than the illustrative names shown here.

6. While doing this, also check whether the installed Copilot SDK requires MCP tool names to be server-qualified (e.g. a `servername__toolname` form) rather than plain unqualified names — adjust every `Tools` /allow-list reference accordingly if so.

If step 4 fails silently (tools listed as available but not actually called), fall back to the item 1 pattern above more heavily — pre-inject data into every specialist rather than relying on live tool calls — until the SDK behavior is confirmed fixed.

11. Specialist responsibilities

MemoryReviewer

Investigate:

- static/global state
- singleton lifetime problems
- unbounded collections
- cache eviction/expiration
- event subscription retention
- timers
- retained closures
- TaskCompletionSource / task retention
- channels/queues

- buffers
- DbContext lifetime
- IDisposable ownership
- request data escaping its intended lifetime

Distinguish:

```
Unintended retention
Allocation pressure
Large-object pressure
Normal long-lived memory
```

CpuReviewer

Investigate:

- nested loops
- algorithmic complexity
- repeated enumeration
- LINQ in loops
- DB calls in loops
- HTTP calls in loops
- serialization in loops
- reflection
- regex-heavy operations
- task creation
- Task.WhenAll fan-out
- Parallel APIs
- retries
- polling

- timers
- lock-heavy code
- GC amplification

Focus on amplification and production-reachable paths rather than isolated micro-optimizations.

ConcurrencyReviewer

Investigate:

- unbounded concurrency
- `Task.WhenAll`
- `Parallel.ForEach`
- `Parallel.ForEachAsync`
- channels
- queues
- `SemaphoreSlim`
- locks
- ThreadPool usage
- overlapping timers
- producer/consumer imbalance
- retry storms

VerificationReviewer

For each strong candidate:

1. attempt to disprove it
2. search for counter-evidence
3. determine whether the behavior is intentional

4. determine the production trigger
 5. determine whether input size/traffic amplifies it
 6. propose or generate a reproducible test/benchmark
 7. optionally correlate with runtime evidence
-

12. Critic / anti-false-positive phase

Every finding must pass a challenge step before appearing in the final report.

For memory findings, search for:

```
Remove
Clear
Dispose
Unsubscribe
Eviction
Expiration
Capacity limits
Cancellation
Periodic cleanup
Replacement
```

For concurrency findings, search for:

```
SemaphoreSlim
bounded Channel
MaxDegreeOfParallelism
batching
rate limiting
cancellation
input size limits
```

For DB/HTTP amplification, search for:

```
batching
bulk APIs
caching
memoization
prefetching
bulk queries
bounded input
```

If counter-evidence exists, downgrade or remove the finding.

13. Evidence levels

Use exactly these evidence levels initially:

```
E0 = heuristic
E1 = static path/ownership evidence
E2 = reproduced by test or benchmark
E3 = corroborated by runtime telemetry/diagnostics
```

Rules:

- E0/E1: call it a **candidate** or **static-analysis finding**
- E2: call it **reproduced behavior**
- E3: call it **runtime-correlated evidence**

Never invent runtime measurements.

Never claim "confirmed production memory leak" without appropriate evidence.

14. Verification tooling

The first version should support verification through executable tests and BenchmarkDotNet where practical.

Possible runtime tools for later integration include:

```
dotnet-counters  
dotnet-trace  
dotnet-dump  
dotnet-gcdump
```

The verifier should not automatically attach to production unless explicit configuration and permissions exist.

For benchmarks, prefer a dedicated test/benchmark project and synthetic controlled inputs.

For performance comparisons, collect at least:

```
elapsed time  
allocation bytes  
GC collection counts where available  
operation count / DB calls / HTTP calls where  
measurable
```

BenchmarkDotNet provides memory/allocation diagnostics through its diagnoser infrastructure. Verify the installed BenchmarkDotNet version before relying on specific exporters or diagnosers.

15. Seeded test repository / evaluation suite

Build a small local benchmark repository containing known cases.

Required positive cases:

```
01-UnboundedCache
02-EventRetention
03-UnboundedChannel
04-DbCallInsideLoop
05-HttpCallInsideLoop
06-TaskWhenAllExplosion
07-BlockingAsync
08-AllocationInsideLoop
09-RetryAmplification
10-OverlappingTimer
```

Required negative/false-positive cases:

```
11-BoundedCache
12-ProperEventUnsubscribe
13-BoundedConcurrency
14-BatchedDatabaseQuery
15-IntentionalStaticState
16-FixedSmallTaskSet
```

The test suite should establish ground truth manually.

Track at minimum:

```
true positives
false positives
```

```
false negatives
verification success
```

Do not stop at tracking these counts. Compute and report **precision** ($TP / (TP + FP)$) and **recall** ($TP / (TP + FN)$) as an explicit, versioned numeric artifact (e.g. `test-assets/results/metrics.json` or a generated markdown table) each time the evaluation suite runs. A described capability to "track false positives" is not the same as a computed number a reviewer can look at.

The goal is not to maximize the number of findings. The goal is to maximize **useful, evidence-backed findings while minimizing false positives**.

15.1 Out-of-sample validation

The seeded repository is built by the same people writing the rules, so passing it is necessary but not sufficient — it does not rule out the rules being overfit to the seeded examples. Once the agent passes the seeded suite, additionally run it against at least one real open-source .NET repository with a documented historical performance issue (for example, a known EF Core N+1 postmortem or GitHub issue) as an independent check that the agent generalizes beyond its own fixtures. Record whether the known issue was found, and at what evidence level.

16. Final report format

The agent should produce clean category sections:

```
# Performance Analysis

Repository: ...
Framework: ...
Analysis timestamp: ...

## Executive Summary

...

## Memory & Allocation

### PERF-001 – Potential unbounded cache
Evidence: E1
Severity: High

Location:
...

Execution path:
HTTP endpoint -> Service -> Cache.Add()

Observed:
...

Why it matters:
...

Counter-evidence considered:
...

Verification:
...

## CPU & Amplification
...

## Concurrency & ThreadPool
```

```
...

## Database / HTTP Amplification
...

## Verification Opportunities
...

## Low-confidence candidates
...
```

Every finding must include:

- stable ID (`FindingId` — see below)
- category
- severity
- evidence level
- exact file/line
- execution path when known
- code mechanism
- trigger
- amplification/lifetime explanation
- counter-evidence
- verification procedure
- remediation direction

FindingId definition: a deterministic hash (e.g. SHA-256, truncated for display) of `ruleId + normalized file path + start line + end line + normalized code snippet`. Normalize the path relative to `RepoPath` and the snippet by trimming whitespace, so the same underlying issue produces the same ID across re-runs even if unrelated parts of the file changed. Two uses depend on this:

- **Deduplication** — if more than one specialist independently surfaces the same underlying issue, the consolidation step (section 10) treats matching `FindingId`s as one finding, not two.
- **Evaluation matching** (section 15) — the automated precision/recall computation joins produced findings to the seeded ground truth by `FindingId` (or by rule ID + expected file/line range, for ground-truth entries defined before a run ever happened), rather than by fuzzy text matching.

Do not provide a numeric "overall score" for the repository in the first version. Favor factual evidence and individual findings.

Include an execution/traceability appendix built from the `subagent.*` lifecycle events captured during orchestration (section 10): which specialist ran, duration, and tool-call count. This supports auditability of how each finding was produced.

Also state the commit SHA the Graphify graph was built from (from `graphify-out/graph.meta.json`, section 4.4) — e.g. "graph current as of commit `abc1234`" — so a reader knows how current the structural context behind the findings is. If the working tree was dirty when the graph was built, say so explicitly (e.g. "graph reflects commit `abc1234` plus uncommitted local changes") rather than silently presenting it as clean.

17. Agent behavior rules

The system prompt for the Performance Agent must enforce:

1. Do not claim a production problem without supporting evidence.
2. Do not call a potential retention path a memory leak automatically.
3. Distinguish retention, allocation pressure and normal memory usage.
4. Prioritize production-reachable paths.
5. Prioritize amplification and repeated execution.
6. Use Graphify for structural reasoning rather than reading the whole repository blindly.
7. Use Roslyn diagnostics as deterministic evidence.
8. Actively search for counter-evidence.
9. Prefer fewer strong findings to many weak findings.
10. Never fabricate traffic, CPU, memory, latency or production telemetry.
11. Do not recommend a micro-optimization unless the affected operation is likely relevant at production scale.
12. State what would verify the hypothesis when runtime evidence is unavailable.

18. Implementation order

Implement in this order:

Milestone 0 — Seeded test repository (build first)

- build the seeded test repository described in section 15 (positive cases 01–10, negative/false-positive cases 11–16) before writing any agent, tool, or analyzer code

- this becomes the fixture every later milestone builds and tests against, rather than something assembled at the end
- commit it as `test-assets/seeded-performance-bugs/` and `test-assets/false-positives/` per the repository structure in section 2
- build its Graphify graph once via a direct `graphify .` command (section 4.1); it is a static fixture and does not need the git-hook refresh mechanism in section 4.4, which exists for the live repository under active development

Milestone 1 — Foundation

- target .NET 10; host the agent as an ASP.NET Core web application (minimal API), not a console app
- stand up the minimal API host, DI container, and configuration (`RepoPath` , `GraphOutputPath` , listen port — section 4.4); this is the foundation Milestone 2's endpoint and worker attach to
- verify current `GitHub.Copilot.SDK` package/API
- run the compatibility probe (section 10.2) before building out the full custom-agent/tool-delegation design; record the result in `docs/architecture.md`
- create agent project
- establish working Copilot session
- establish repository read/search capability

Milestone 2 — Graphify

- install/pin Graphify
- build graph
- run MCP server
- connect Graphify MCP to Copilot session

- validate `query_graph`, `get_node`, `get_neighbors`, `shortest_path`
- install Husky.Net, add the MSBuild auto-install target, and configure the `post-commit` hook/task (section 4.4)
- implement `GraphifyUpdater`, the `/git-post-commit` endpoint, and the `GraphRefreshWorker` background consumer
- implement the startup and analysis-start self-healing freshness checks

Milestone 3 – Roslyn analyzer

- create analyzer project
- implement PERF001–PERF006
- create analyzer test project
- make analyzer build reproducibly

Milestone 4 – Static-analysis tool

- implement `run_static_analysis`
- normalize diagnostics to JSON
- add analyzer results to Copilot context

Milestone 5 – Orchestrated review

- implement the `CustomAgentConfig` block for all four specialists – `MemoryReviewer`, `CpuReviewer`, `ConcurrencyReviewer`, and `VerificationReviewer` (section 10) – including the shared `specialistTools` list and `Infer = false` on `VerificationReviewer`
- wire `DefaultAgent.ExcludedTools` on the orchestrator
- subscribe to `subagent.*` lifecycle events and record them for the traceability appendix (section 16)

- evidence/candidate consolidation across the three analysis reviewers, ready to hand to VerificationReviewer

Milestone 6 – Critic

- VerificationReviewer already exists as of Milestone 5; this milestone is about invoking it, not building a separate component
- orchestrator-side explicit invocation of VerificationReviewer once candidates are consolidated
- counter-evidence search prompt tuning (section 12)
- downgrade/remove unsupported findings

Milestone 7 – Verification

- test generation where practical
- BenchmarkDotNet integration
- evidence-level classification

Milestone 8 – Evaluation suite

- run the full agent against the seeded repository and false-positive cases built in Milestone 0
 - automated result comparison against ground truth, including the precision/recall computation in section 15
 - out-of-sample validation against a real open-source repository (section 15)
 - regression tests
 - capture the canned/replayed transcript for the demo fallback path (section 23) once the pipeline above is passing
-

19. Acceptance criteria

The implementation is complete for version 1 when all of the following are true:

1. The Copilot agent can analyze the configured repository via the `POST /analyze` entry point (section 10.1).
2. Graphify MCP is connected and usable from the same Copilot session, and its graph is kept current automatically via the git post-commit hook and the self-healing freshness checks (section 4.4).
3. `run_static_analysis` returns structured Roslyn diagnostics and is reachable by the specialist sub-agents that call it (section 10).
4. PERF001–PERF006 have automated analyzer tests.
5. The agent correlates Roslyn findings with Graphify paths.
6. Memory, CPU, and Concurrency specialist sub-agents can run, and VerificationReviewer can be invoked explicitly as the critic/verifier phase (section 10, Milestone 6).
7. The critic/verifier phase can downgrade unsupported findings.
8. Final findings contain exact evidence and file/line locations, the execution/traceability appendix, and the graph's source commit SHA (section 16).
9. The agent does not invent runtime measurements.
10. Seeded positive and negative examples exist, and precision/recall are computed and reported against them (section 15).
11. At least one out-of-sample validation run against a real open-source repository has been performed and its result recorded (section 15.1).

12. A canned/replayed transcript of a full successful run exists as a demo fallback (section 23).
 13. The project can be built and executed without CodeQL.
 14. The implementation works in the intended offline/private repository environment except for the configured Copilot model/service dependency.
-

20. Non-goals for version 1

Do not initially build:

- a full replacement for a commercial static-analysis platform
- a complete interprocedural compiler/data-flow framework
- automatic production debugging without explicit runtime access
- automatic source rewriting/fixes for every finding
- CodeQL integration
- dozens of low-value micro-optimization rules
- an all-language analyzer
- multi-repository analysis in a single running instance (section: Scope decision) — v1 is a local companion service for one configured repository

Focus on high-value C#/.NET production performance problems.

21. Engineering constraints

- Prefer existing repository conventions.
 - Do not introduce unnecessary dependencies.
 - Pin versions where reproducibility matters.
 - Make paths/configuration work on Windows and Linux where practical.
 - Do not assume Python is on PATH; make Graphify configuration explicit and configurable.
 - Do not hard-code user-specific paths.
 - Keep MCP Graphify read-only.
 - Do not allow the performance analyzer to modify application source during analysis.
 - Add cancellation and timeouts to external process execution.
 - Sanitize/limit analyzer output before passing it to the LLM.
 - Log tool failures separately from analysis findings, including `graphify` refresh failures from the background worker (section 4.4).
 - Bind the `/git-post-commit` refresh endpoint to loopback only (`127.0.0.1 / localhost`) – it is a local development trigger, not a public API.
-

22. Developer notes

The GitHub Copilot SDK currently supports .NET and uses the Copilot CLI engine underneath the SDK. The .NET package is `GitHub.Copilot.SDK`. The SDK supports custom agents, tools, skills and MCP servers. [GitHub Copilot SDK](#)

The Copilot SDK MCP documentation currently supports local/stdio MCP servers and tool allow-lists. [GitHub Docs –](#)

[MCP](#)

Graphify currently documents an MCP server exposing structured graph traversal and local AST extraction. [Graphify Labs](#)

Roslyn analyzers are compiler-integrated custom diagnostics based on `DiagnosticAnalyzer` and related APIs. [Microsoft Learn — Roslyn analyzer tutorial](#)

When implementing, prefer the exact API surface of the versions present in the repository rather than copying stale snippets from this specification.

23. Demo reliability (fallback path)

Maintain a canned/replayed transcript of a full successful run against the seeded repository (Milestone 0), captured once the pipeline is working end to end, as a fallback for live demonstration.

If the live Copilot session is slow, rate-limited, or unavailable during a time-boxed demo, replay the canned transcript and final report instead of running the pipeline live. This is insurance against external-service variability during judging, not a replacement for having a working live pipeline.

Keep the fallback transcript current: regenerate it whenever the seeded repository, rule set, or report format changes materially, so it does not drift from what the live agent actually produces.